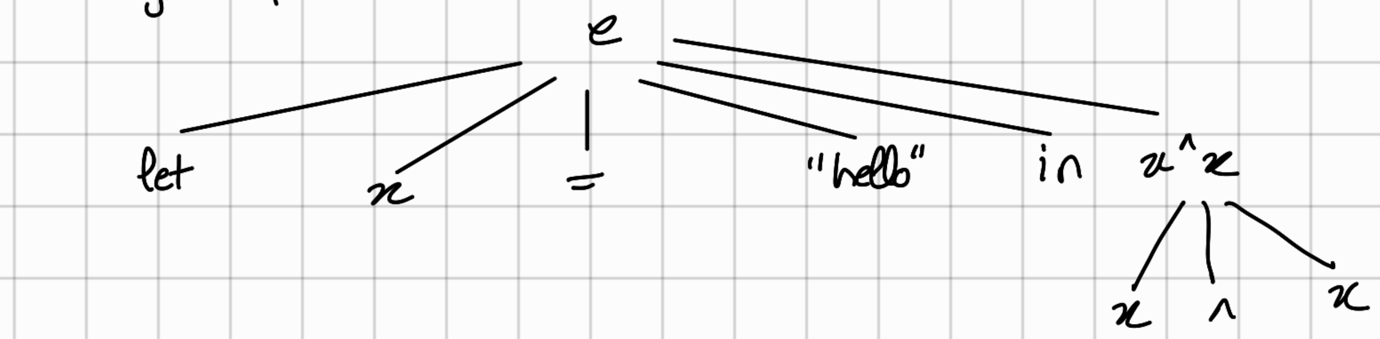
TD1 : Grammaires.

Exercice 1-)

Q.1-) Écrire l'arbre syntaxique des expressions suivantes :

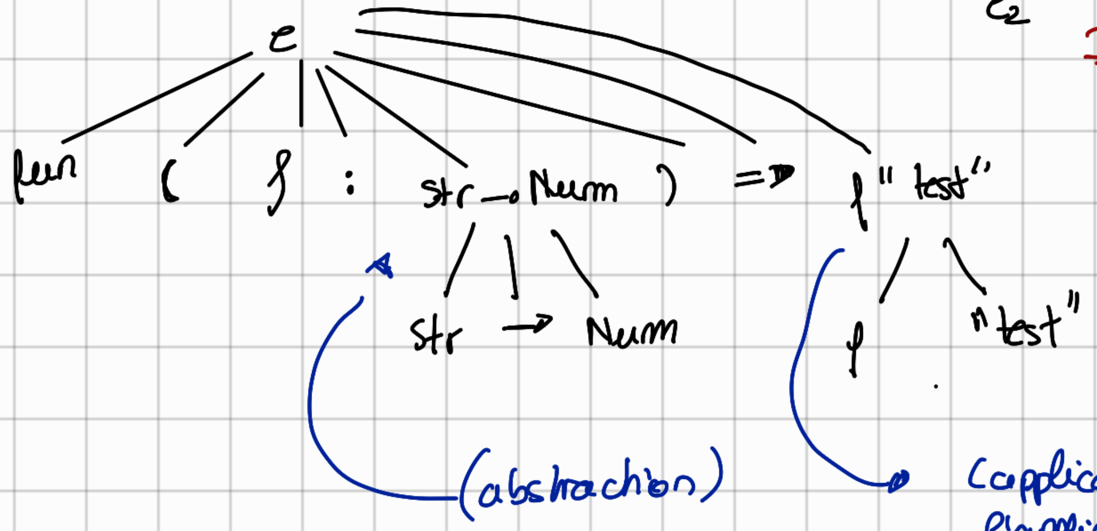
$e : \text{let } \underbrace{x}_{x} = \underbrace{\text{"hello"}}_{e_1} \text{ in } \underbrace{x \,\hat{}\, x}_{e_2} .$ — c'est une expression (définition)

Arbre syntaxique associé :



$e : \text{fun } (f : \text{Str} \rightarrow \text{Num}) \Rightarrow (\underbrace{f}_{e_1} \; \underbrace{\text{"test"}}_{e_2}).$ → c'est une expression

Il y a ambiguïté
L'application à droite est prioritaire sur l'abstraction.

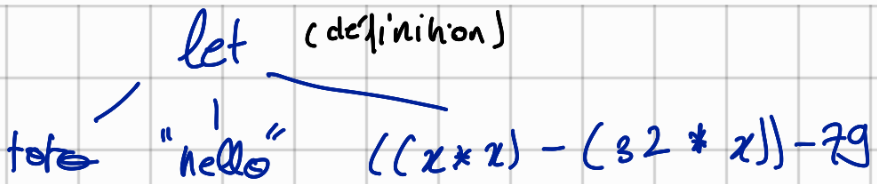


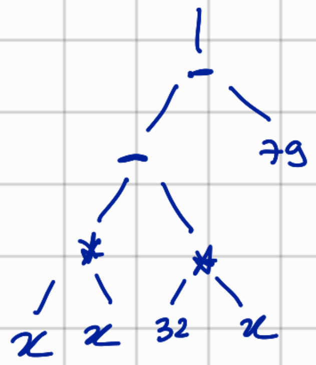
(application) ; on définit l'application comme étant séparée par un espace

Q.2-) Écrire l'AST des expressions suivantes.
Expliciter les associativités et priorités dont vous avez besoin.

$e : \text{let } \underbrace{\text{toto}}_{x} = \underbrace{\text{"hello"}}_{e_1} \text{ in } \underbrace{((x * x) - (32 * x)) + 79}_{e_2} .$

c'est une règle de définition donc on étiquette par let :





on a utilisé l'associativité à gauche pour - ; et * prioritaire sur -

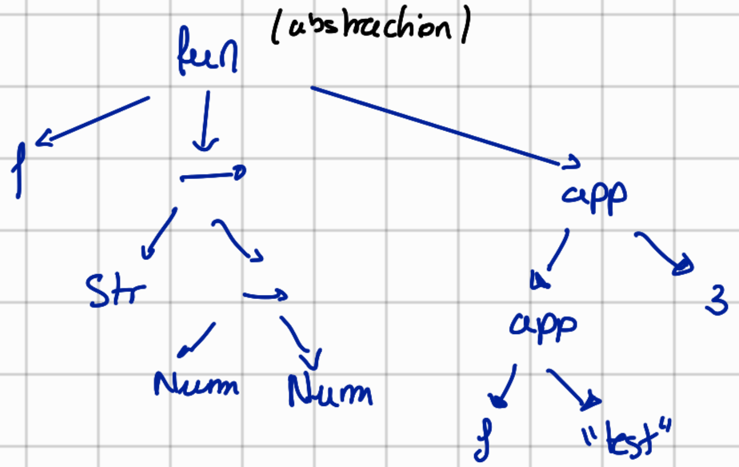
e : fun (f : Str → (Num → Num)) ⇒ ((f "test") 3)

application prioritaire sur abstraction

Les flèches : associativité à droite
Les applications : associativité à gauche

Ex: $f: a \to (b \to (c \to d))$
$((f\ a)\ b)\ c$

e est une règle d'abstraction donc sera étiquetée par fun :



fun a besoin d'un nom, type et d'une expression

Les Nœuds internes correspondent aux règles (+, -, ...)

Exercice 2-)

Q.1-) Les ... ne seraient pas acceptés dans une grammaire formelle. Trouver une manière d'exprimer le produit formel+.

Type e ::= Num
⋮

$\vdots$
$(\Gamma_1, \ldots, \Gamma_n)$
$\vdots$

Expression $e ::= \vdots$
$(e_1, \ldots, e_n)$
$\vdots$

→ TYPOLIST $\ell ::= \Gamma$ (ou Vide)
Γ, ℓ

→ Même chose pour les Expressions.

TYPE $\Gamma ::=$ Num
Str
$\Gamma_1 \rightarrow \Gamma_2$
(ℓ)
unit

Q.2-) Le test est limité à la comparaison avec 0. Généraliser le test en ajoutant les booléens avec (certains de) leur opérateur.

$e ::=$ if b then e_2 else e_3 (test)

BOOL2 $b ::=$ true
false
$b_1 \wedge b_2$ (et)
$b_1 \vee b_2$ (ou)
$\neg b$ (nég)
$e_1 \geqslant e_2$ (comparaison)

GRAMMAIRE = définition concrète et récursive d'un objet.

Q.3-) En Ocaml, les déf° let ... in sont plus générales : on peut remplacer la variable, soit un tuple de motifs (pouvant être

vide) ; soit un joker _.
Généraliser le let ... in

motifs (pattern)

Expression

$e ::=$ ⋮
$\quad \text{let } p = e_1 \text{ in } e_2$
⋮

MOTIFS $p ::= x$
$\quad (p_1, \dots, p_n)$
$\quad \_$

denoureis @ lipn . univ - paris13 . fr